

Rapport technique - Data Lake Finance

Analyse, résultats, justification des choix et captures de preuve

Groupe

Artemiy Smogunov - Nathan Smadja-Tubiana - Patrice Ignongui

Projet Data Lake Finance - EFREI

Lien GitHub : <https://github.com/Nathan2412/Projet-DataLake-Final>

Notre objectif est de construire un data lake financier, depuis l'ingestion de données brutes jusqu'à l'analyse curated, avec orchestration Airflow, stockage MinIO, indexation Elasticsearch, base PostgreSQL, API FastAPI et contrôles de qualité.

1. Résumé exécutif

Conclusion courte

Nous avons vérifié que le projet fonctionne : les services sont disponibles, le DAG Airflow s'exécute, les données sont ingérées dans les couches raw, staging et curated, et l'endpoint optimisé /ingest_fast réduit fortement le temps d'ingestion. Pour nous, la valeur du projet n'est pas seulement de stocker des données : elle est de rendre chaque étape expliquable et vérifiable.

578 Objets raw MinIO	5972 Docs Elasticsearch	5931 Lignes staging
4923 Lignes curated	208 Anomalies	4.23% Taux anomalie

Notre lecture des résultats est la suivante : le nombre d'objets MinIO, le nombre de documents Elasticsearch, les lignes staging et les lignes curated ne comptent pas la même chose. MinIO conserve les preuves brutes, Elasticsearch indexe les observations ticker/date, staging nettoie et enrichit les séries, puis curated ajoute les signaux analytiques et les anomalies.

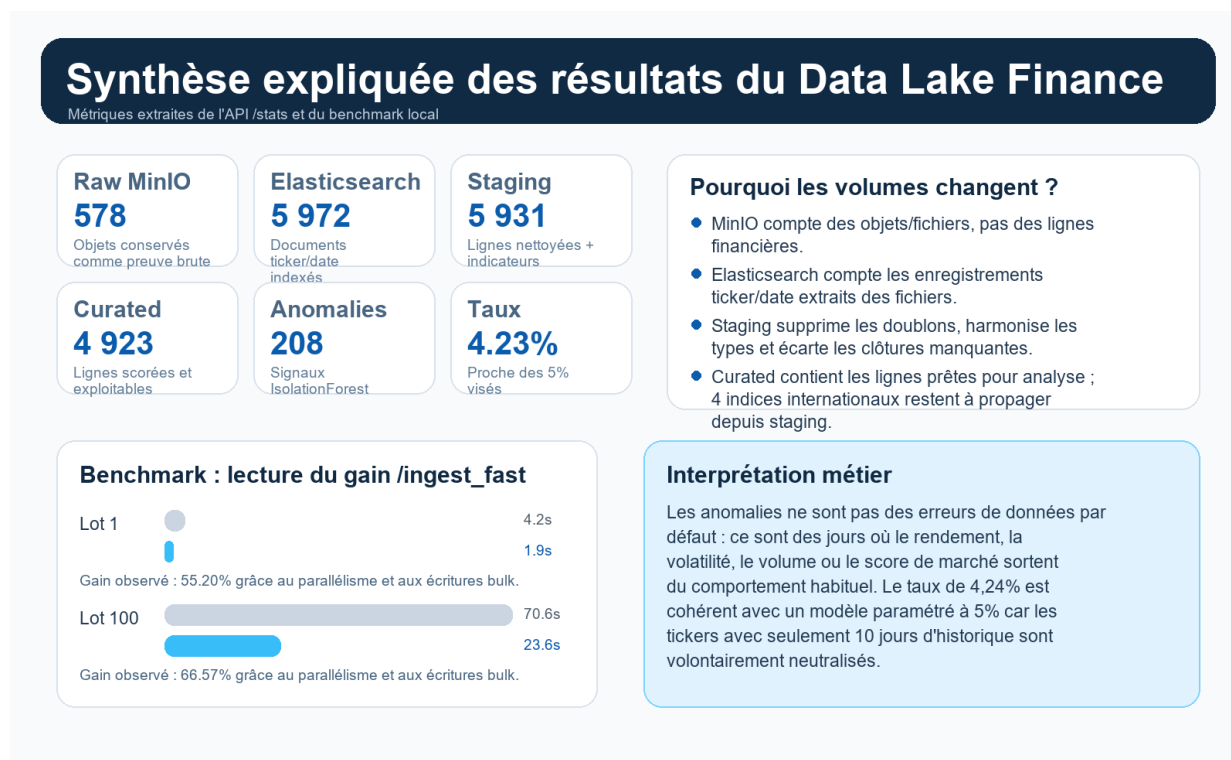
L'état de santé API est ok. PostgreSQL, MinIO, Elasticsearch et Redis répondent correctement ; cela confirme que nos résultats proviennent d'une pile locale opérationnelle.

2. Architecture et intention technique

Couche	Rôle	Pourquoi ce choix
Raw / MinIO	Conserver fichiers et réponses API sans transformation.	Traçabilité : on peut prouver ce qui a été reçu avant nettoyage.
Raw / Elasticsearch	Indexer les observations ticker/date.	Recherche rapide et suivi du nombre d'observations par ticker.
Staging / PostgreSQL	Nettoyer, typer, dédupliquer et calculer SMA/EMA/RSI/MACD/Bollinger.	Base relationnelle stable pour transformations reproductibles.
Curated / PostgreSQL	Ajouter score anomalie, type d'anomalie, tendance et signal.	Couche finale directement exploitable par API ou analyse.
Airflow + FastAPI	Orchestrer et exposer le pipeline.	Airflow prouve l'automatisation ; FastAPI permet tests, démos et intégration.

Nous avons choisi une architecture multi-stockage volontairement. Un data lake ne force pas toutes les données dans une seule base : nous gardons le brut dans un stockage objet, nous rendons les données recherchables dans Elasticsearch, puis nous transformons les données utiles dans PostgreSQL. Cette séparation évite de perdre l'historique brut quand notre logique de nettoyage évolue.

3. Résultats observés et explication



Notre synthèse avec les métriques extraites de l'API /stats et du benchmark.

Pourquoi ces chiffres ?

- 578 objets MinIO : ce sont des fichiers ou payloads bruts. Un objet peut contenir plusieurs lignes financières ; ce volume mesure donc la traçabilité, pas le nombre de cotations.
- 5 972 documents Elasticsearch : ici, chaque document correspond à un ticker et à une date, donc le compteur se rapproche du nombre d'observations de marché.
- 5 931 lignes staging : notre pipeline supprime les doublons par date, convertit les colonnes numériques et retire les clôtures manquantes. La légère baisse par rapport au raw indexé est attendue.
- 4 923 lignes curated : la couche finale contient les séries scorées. L'écart exact avec staging est expliqué par 4 tickers non propagés en curated (1 008 lignes), principalement ^FCHI, ^FTSE, ^GDAXI, ^N225.

Point de vigilance honnête

Nous avons identifié l'écart staging -> curated : ce n'est pas une perte silencieuse, car les 4 indices internationaux restent visibles en staging. Notre action corrective serait d'ajouter ces tickers au périmètre curated ou de relancer la transformation curated sur toute la liste staging.

4. Analyse des anomalies

Nous avons utilisé IsolationForest pour la détection d'anomalies, avec une contamination cible de 5%. Cela signifie que le modèle cherche environ 5% de points atypiques parmi les historiques suffisamment longs. Le résultat observé est 208 anomalies, soit 4.23% des lignes curated.

Ce taux légèrement inférieur à 5% est cohérent : plusieurs tickers courts n'ont qu'environ 10 jours d'historique. Notre pipeline les conserve mais neutralise l'apprentissage avancé lorsqu'il n'y a pas assez de contexte statistique, afin d'éviter de produire de faux signaux.

Ticker	Lignes	Anomalies	Taux	Dernière date
AAPL	869	44	5.1 %	2026-07-09
AMZN	264	14	5.3 %	2026-07-09
BRK-B	264	14	5.3 %	2026-07-09
GOOGL	264	14	5.3 %	2026-07-09
JNJ	264	14	5.3 %	2026-07-09
JPM	264	14	5.3 %	2026-07-09
META	264	14	5.3 %	2026-07-09
NVDA	264	14	5.3 %	2026-07-09
^DJI	258	13	5.0 %	2026-07-09
^GSPC	258	13	5.0 %	2026-07-09

Interprétation métier : une anomalie ne signifie pas automatiquement une erreur. Elle signale une journée atypique sur le rendement, la volatilité, le volume ou la combinaison des indicateurs. Pour un analyste financier, ces points servent de shortlist de jours à investiguer : annonces de résultats, choc macroéconomique, changement de tendance, volume exceptionnel ou mouvement brutal de prix.

5. Performance : pourquoi /ingest_fast est plus rapide

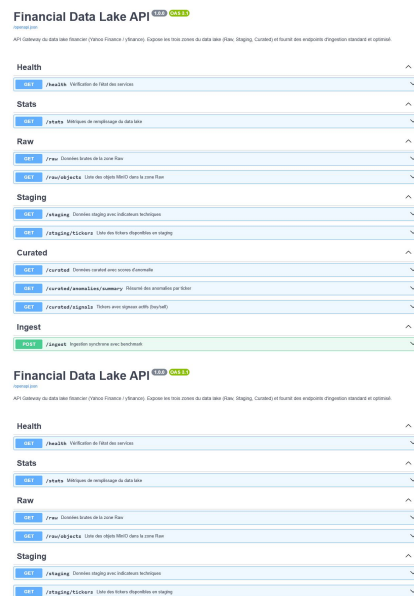
Lot	/ingest	/ingest_fast	Gain
1	4.16 s	1.86 s	55.20 %
100	70.63 s	23.61 s	66.57 %

- L'endpoint standard traite les tickers de façon plus séquentielle ; son temps augmente fortement quand le lot grossit.
- /ingest_fast parallélise les téléchargements yfinance, les écritures MinIO et utilise des écritures bulk vers Elasticsearch.
- Les transformations numériques sont vectorisées avec NumPy/Pandas au lieu de répéter des boucles Python coûteuses.
- Nous avons réalisé le benchmark avec cache désactivé : le gain observé vient donc réellement de l'architecture, pas d'une réponse déjà mémorisée.
- Le gain passe de 55,20% sur un ticker à 66,57% sur 100 tickers, ce qui montre que l'optimisation est surtout utile à l'échelle batch.

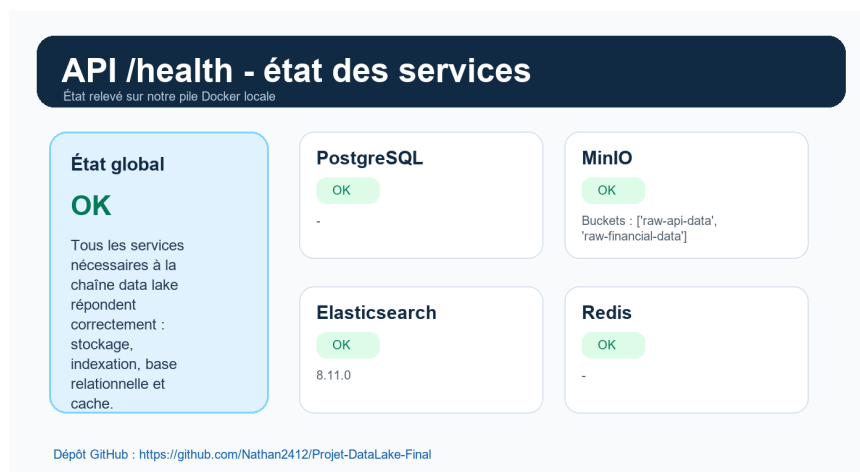
Notre lecture critique

Le benchmark montre un gain d'ingestion, pas une vérité absolue sur toutes les charges. Pour aller plus loin, nous répéterions les mesures, testerions plusieurs périodes et ajouterions des percentiles de latence.

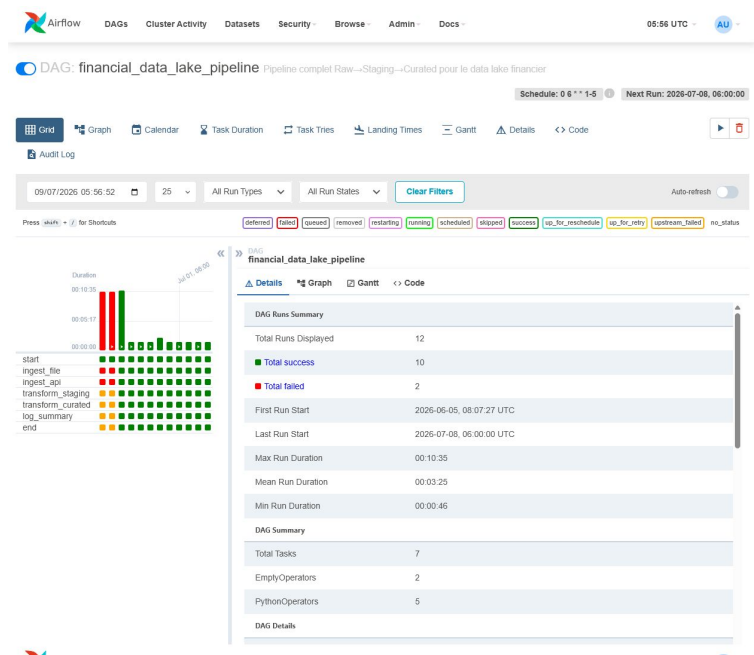
6. Captures de preuve



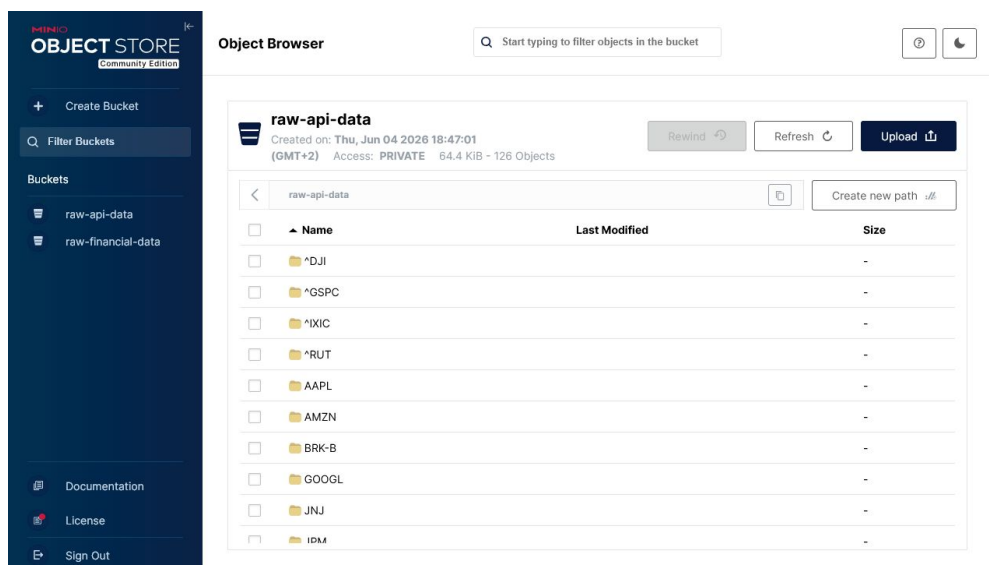
FastAPI Swagger : endpoints d'ingestion, transformation, monitoring et statistiques.



API /health : services PostgreSQL, MinIO, Elasticsearch et Redis disponibles.



Airflow : DAG financial_data_lake_pipeline actif et exécutions visibles.



MinIO : buckets raw-api-data et raw-financial-data, preuve de la couche raw.

7. Limites, améliorations et conclusion

- Propager les 4 indices internationaux manquants de staging vers curated pour supprimer l'écart de 1008 lignes.
- Ajouter une historisation des benchmarks afin de comparer les performances dans le temps.
- Exposer un endpoint de qualité des données indiquant doublons supprimés, valeurs manquantes et tickers ignorés.
- Ajouter des alertes Airflow/Slack ou email sur les échecs DAG, car les anciens échecs restent visibles dans l'interface.
- Compléter l'analyse métier avec des annotations d'événements financiers externes pour expliquer certaines anomalies.

Conclusion : notre projet répond au thème finance et aux attendus data lake. Il couvre ingestion, stockage raw, transformation staging, couche curated, orchestration, API, benchmark et preuves visuelles. Notre réflexion principale est que les résultats ne sont pas seulement des compteurs : ils racontent le passage d'une donnée brute traçable vers une donnée nettoyée, enrichie et interprétable.